

A MINI PROJECT REPORT
ON
REPOSYS – REPROGRAPHY
AUTOMATION SYSTEM

Submitted by

JEFRI JIJI

(Register Number: MGP22NMC034)

Project Team 04 with:

JERIN SEBASTIAN (MGP22NMC037) • SETHULAKSHMI P.S
(MGP22NMC050)

to the

APJ ABDUL KALAM TECHNOLOGICAL
UNIVERSITY

in partial fulfilment of the requirements for the award of the degree of

MASTER OF COMPUTER APPLICATIONS
(INTEGRATED)



DEPARTMENT OF COMPUTER APPLICATIONS
SAINTGITS COLLEGE OF ENGINEERING
(AUTONOMOUS)

PATHAMUTTOM, KOTTAYAM, KERALA – 686532

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this mini project report entitled “**REPOSYS – REPROGRAPHY AUTOMATION SYSTEM**” is a bonafide record of the work carried out by **JEFRI JIJ** (Register Number: **MGP22NMC034**) in partial fulfilment of the requirements for the award of the degree of **Master of Computer Applications (Integrated)** of APJ Abdul Kalam Technological University under our guidance and supervision during the academic year 2026. This report has not been submitted, in whole or in part, for the award of any other degree or diploma to any university or institution.

[GUIDE NAME]

Project Guide
Department of Computer Applications
Saintgits College of Engineering

[COORDINATOR NAME]

Project Coordinator
Department of Computer Applications
Saintgits College of Engineering

[HOD NAME]

Head of the Department
Department of Computer Applications
Saintgits College of Engineering

[PRINCIPAL NAME]

Principal
Saintgits College of Engineering
(Autonomous)
Kottayam

ACKNOWLEDGEMENT

First and foremost, I offer my humble gratitude to Almighty God for showering His divine blessings and wisdom, which guided me through each stage of this project development and report writing.

I express my profound gratitude to the Management of **Saintgits College of Engineering (Autonomous)**, Kottayam, for providing excellent academic infrastructure, modern laboratory resources, and a supportive learning atmosphere conducive to technological exploration.

I place on record my sincere thanks to our respected Principal, **[PRINCIPAL NAME]**, for his leadership, encouragement, and institutional backing throughout our Integrated MCA programme.

I express my deepest appreciation to **[HOD NAME]**, Head of the Department of Computer Applications, for his visionary guidance, constructive feedback, and continuous motivation during the conceptualisation and execution of this work.

I am immensely indebted to our Project Coordinator, **[COORDINATOR NAME]**, for meticulously planning the academic reviews, providing structured schedules, and ensuring high engineering benchmarks at every milestone.

I extend my heartfelt gratitude to my Project Guide, **[GUIDE NAME]**, whose patient supervision, invaluable technical insights, and rigorous evaluation played a pivotal role in refining both the architecture of REPOSYS and this documentation.

I also extend my sincere thanks to all faculty members, technical staff, and reprography operators of the institution whose daily workflows and practical feedback shaped the functional specifications of this system.

Finally, I express my warmest thanks to my project teammates Jerin Sebastian and Sethulakshmi P.S, my classmates, and my beloved parents and family members whose unwavering moral support and encouragement enabled the successful completion of this endeavor.

ABSTRACT

Academic reprography centres in higher education institutions serve hundreds of students, faculty, and administrative staff daily. Despite advances in campus digitisation, reprography operations have historically remained manual, paper-reliant, and inefficient. Users encounter severe physical queue congestion, privacy vulnerabilities caused by sharing confidential academic documents across personal WhatsApp channels, transaction friction due to exact cash change requirements, and lack of visibility into print job progression. Conversely, reprography counter operators face chaotic order queues, untracked paper waste from abandoned printouts, and inaccurate manual billing.

REPOSYS (Reprography Automation System) is an enterprise-grade, cloud-enabled web platform and hardware-integrated automation ecosystem designed to digitise the end-to-end reprography lifecycle. Architected using the modern MERN stack (MongoDB, Express.js, React 19, Node.js), REPOSYS introduces a three-tier role-based operational framework serving Students, Faculty, Counter Staff, Administrators, and Guests. The system incorporates an automated document analysis pipeline featuring client-side and server-side MIME-type validation, page-count extraction, and luminance-based blank-page detection.

A central innovation of REPOSYS is its **Composite Priority Queue Algorithm with Dynamic Aging Deduction**.

To prevent student starvation while respecting academic hierarchy, the scheduling engine calculates effective priority by combining role weights, job complexity, and an automated aging deduction function that systematically elevates long-waiting orders. Transactions are managed through integrated online Razorpay gateways, an internal digital wallet backed by MongoDB ACID transactions, and a Pay at Counter (PAC) workflow with automated timeout safeguards. Physical printing is bridged via a dedicated **Electron-based Windows Print Agent** that connects local hardware to cloud dispatchers using bi-directional Socket.IO pipelines.

Progressive Web App (PWA) capabilities enable offline queue status caching and Web Push alerts. Comprehensive quality assurance using an automated Selenium WebDriver test suite confirmed 114 passing test suites out of 120 rigorous operational cases (95.0% passing rate). REPOSYS replaces chaotic counter queues with an auditable, cashless, transparent, and eco-friendly digital campus infrastructure.

TABLE OF CONTENTS

Chapter	Title	Page No.
	BONAFIDE CERTIFICATE	ii
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
	LIST OF TABLES	vii
	LIST OF FIGURES	viii
1	INTRODUCTION	1
	1.1 Introduction	1
	1.2 Objectives of the Project	3
	1.3 Scope and Availability	5
2	REQUIREMENTS AND ANALYSIS	7
	2.1 Existing System / Problem Statement	7
	2.2 Proposed System / Solution Overview	10
	2.3 Feasibility Study	12
	2.4 Conceptual Modelling	15
	2.5 Planning and Scheduling	17
3	SYSTEM SPECIFICATION	20
	3.1 Software and Hardware Requirements	20

	3.2 Functional Specifications	23
	3.3 Tools and Platforms Used	26
	3.4 Development Environment	28
4	SYSTEM DESIGN	30
	4.1 Module Descriptions	30
	4.2 Data / Schema Design	34
	4.3 Procedural / Flow Design	37
	4.4 User Interface Design	42
5	DEVELOPMENT METHODOLOGY	44
	5.1 Project Roadmap	44
	5.2 User Stories	46
	5.3 Test Plan	48
6	IMPLEMENTATION AND TESTING	50
	6.1 Implementation Procedure	50
	6.2 Testing Methods and Results	54
7	CONCLUSION	58
	7.1 Limitations	58
	7.2 Future Scope	59
	7.3 Conclusion	60
8	APPENDIX	61
	8.1 Supporting Documentation	61
	8.2 Sample Code / Queries	62

	8.3 User Interface Screenshots	65
	8.4 Git Logs	71
9	REFERENCES	73

LIST OF TABLES

Table No.	Title	Page No.
Table 2.1	System Comparison: Existing Manual System vs. Proposed REPOSYS	11
Table 2.2	Official Project Schedule and Milestone Tracking (Scrum Register 20IMCAP501)	18
Table 3.1	Production Software Stack and Dependency Specifications	21
Table 4.1	Database Schema Specifications of Core REPOSYS Collections	35
Table 6.1	Verified Feature Implementation Matrix across REPOSYS	52
Table 6.2	Representative Automated and Functional Test Results	55
Table 8.1	Verifiable Version Control Commit History (Git Logs)	71

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 2.1	Process Flow and Bottlenecks of the Existing Manual System	9
Figure 2.2	Level 0 Context Data Flow Diagram of REPOSYS	15
Figure 2.3	Level 1 Functional Data Flow Diagram of the Complete Order Pipeline	16
Figure 4.1	Automated Shop Scheduling Logic and Operational State Transitions	39
Figure 4.2	System Use Case Diagram: User, Staff, and Admin Operational Boundaries	40
Figure 4.3	System Activity Diagram: Document Upload, Analysis, and Production Flow	41
Figure 4.4	System Sequence Diagram: Real-Time Event Dispatching and Queue Updates	42
Figure 8.1	REPOSYS Public Landing and Portal Gateway	65
Figure 8.2	Secure User Authentication and Multi-Role Login Portal	66
Figure 8.3	Student and Faculty Account Registration Portal	66

Figure 8.4	System Architecture Overview and About REPOSYS Portal	67
Figure 8.5	Reprography Operational Inquiries and Support Contact Portal	67
Figure 8.6	Account Recovery and Password Reset Workflow	68
Figure 8.7	Automated Selenium Test Run: Student Order Workflow	69
Figure 8.8	Automated Selenium Test Run: Admin Role Route Access	69
Figure 8.9	Automated Selenium Test Run: Counter Staff Queue Access	70

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

In contemporary tertiary educational institutions, academic reprography and document management centres represent indispensable utility hubs. Every working day, students, research scholars, teaching faculty, and administrative personnel generate substantial demands for printing lecture notes, laboratory observation manuals, seminar presentations, dissertations, project documentation, question papers, and institutional circulars. Despite rapid campus digitisation across learning management systems (LMS) and enterprise resource planning (ERP) portals, the reprography service tier has remained an acute operational bottleneck, heavily entrenched in legacy manual practices.

In the conventional manual operating model, students are required to physically travel to the campus reprography centre, often enduring long queues during morning peak hours and pre-examination submission deadlines. Submitting electronic files frequently relies on unstandardised, insecure workarounds: students either transfer documents using uninspected USB flash drives—introducing significant malware vectors into counter workstations—or transmit files over consumer messaging platforms such as WhatsApp or Telegram. These manual practices give rise to critical vulnerabilities:

1. **Privacy Infringement and Data Exposure:** Submitting personal assignment files, identity cards, or research papers over personal mobile messaging channels exposes phone numbers and personal documents to counter staff and third-party chat databases without institutional access auditing.
2. **Physical Counter Congestion:** Students must remain physically co-located at the reprography counter throughout the entire processing duration—from file retrieval and configuration to printing, binding, and billing—resulting in lobby overcrowding and lost instructional time.
3. **Transaction and Billing Inefficiencies:** Cash-based counter transactions encounter persistent friction due to shortages of low-denomination physical currency notes and coins. Manual calculation of complex multi-page, double-sided, and binding combinations frequently results in billing inaccuracies and revenue leakage.
4. **Paper Waste and Abandoned Output:** Due to communication disconnects, print operators frequently print jobs that users subsequently fail to collect, or produce incorrect orientations and color modes, leading to high volumes of abandoned paper waste and unrecoverable toner expenditure.
5. **Absence of Prioritisation for Academic Emergencies:** Urgent examination-related printing requisitions from academic faculty are subjected to identical physical First-In, First-Out (FIFO) queue bottlenecks alongside routine student printouts, compromising institutional timeliness.

To decisively resolve these systemic challenges, **REPOSYS (Reprography Automation System)** was conceptualised, designed, and implemented as a comprehensive, cloud-native web platform and hardware-interfaced automation ecosystem. Developed as an Integrated Master of Computer Applications (IMCA) capstone initiative at Saintgits College of Engineering (Autonomous), REPOSYS bridges academic document creators, reprography staff, institutional administrators, and physical printing machinery through an auditable, cashless, and highly responsive operational framework.

REPOSYS leverages modern software engineering standards, employing a three-tier role-based operational architecture built on the MongoDB, Express.js, React 19, and Node.js (MERN) stack. It incorporates cloud object storage via Cloudinary, secure cryptographic transaction validation with Razorpay and internal digital wallets, bi-directional event distribution via Socket.IO, an automated Progressive Web App (PWA) client, and an autonomous desktop Print Agent communicating with Windows spooler sub-systems.

1.2 OBJECTIVES OF THE PROJECT

The overarching mission of REPOSYS is to replace error-prone, manual campus document handling with a modern, dependable, and user-centric digital reprography platform. To realise this vision, the engineering objectives are classified across four functional domains:

1.2.1 Operational and Workflow Objectives

1. **Decoupled Document Submission:** To enable authenticated campus users and temporary kiosk guests to upload documents remotely from any browser, smartphone, or terminal, eliminating physical presence during job queuing.
2. **Automated Document Analysis and Pre-flight Inspection:** To implement automated server-side file analysis that extracts page counts, detects color profiles, validates MIME signatures, and identifies blank pages prior to job confirmation, eliminating user error and incorrect billing.
3. **Dynamic Priority Queue Scheduling:** To establish an equitable, multi-factor priority queue engine that balances urgent faculty requirements against student fairness through an automated aging deduction algorithm, preventing job starvation.
4. **Secure Two-Factor Physical Handover:** To implement One-Time Password (OTP) verification and QR-based collection mechanisms at the counter, ensuring that documents are delivered exclusively to authorised owners.

1.2.2 Financial and Transactional Objectives

1. **Unified Multi-Modal Payments:** To support online UPI/Card transactions via Razorpay, an internal pre-funded student digital wallet with ACID guarantees, and regulated Pay at Counter (PAC) cash workflows.
2. **Collaborative Cost Sharing:** To provide an integrated split-payment mechanism enabling group members to disburse printing costs for shared academic projects atomically across individual student wallets.

3. **Automated Financial Ledgering and Auditing:** To maintain an immutable, append-only transaction ledger and exportable financial reports for institutional administrative reconciliation.

1.2.3 Hardware and Architectural Integration Objectives

1. **Autonomous Physical Spooling via Print Agent:** To bridge cloud backend dispatchers with local counter hardware through a specialised Electron-based Windows Print Agent that automates driver selection and document spooling.
2. **Real-Time Operational Transparency:** To broadcast bi-directional lifecycle events (e.g., job placement, printing progress, completion notifications, queue wait-time adjustments) to connected clients without browser polling.
3. **Automated Shop Scheduling:** To run background cron services enforcing campus operational hours (9:00 AM to 5:00 PM IST) while managing payment timeouts and uncollected order archival.

1.2.4 Sustainability and Security Objectives

1. **Resource Conservation and Green Computing:** To reduce paper and toner wastage through pre-print configuration summaries, blank-page warnings, and optimized double-sided layout incentives.
2. **End-to-End Cryptographic Security:** To enforce strict role-based access control (RBAC), JSON Web Token (JWT) session security, Cloudinary signed URL access, and data protection against unauthorized document disclosures.

1.3 SCOPE AND AVAILABILITY

The operational scope of REPOSYS encompasses all primary document processing, queue organisation, financial settlement, physical printing, and administrative reporting requirements of an academic campus.

1.3.1 User Roles and Stakeholder Scope

The system establishes clearly demarcated functional boundaries for five distinct user personas:

1. **Student Persona:** Access to multi-document upload wizards, real-time cost estimation, wallet balance management, friend lists, split billing, live queue tracking, complaint lodgement, and PDF receipt downloads.
2. **Faculty Persona:** Priority submission channels for academic and examination materials, customized delivery notes, departmental billing allocation, and direct counter priority.
3. **Counter Staff Persona:** Operational terminal displaying active priority queues, manual cash collection controls, physical print dispatch triggers, stock inventory updates, and OTP-based pickup validation.
4. **Administrator Persona:** Institutional oversight including dynamic price-per-page configuration, user account activation/restriction, system configuration, audit log inspection, and aggregate financial reporting.
5. **Guest / Kiosk Persona:** Frictionless, registration-free terminal access via time-bound guest tokens and QR code tracking, with automated session expiration for campus visitors and parents.

1.3.2 Physical and Hardware Scope

REPOSYS is designed to interface with standard multi-function commercial printers (MFPs), heavy-duty laser printers, and comb/spiral binding stations installed within the institutional reprography facility. Network connectivity encompasses the campus local area network (LAN), Wi-Fi, and public Internet access for remote student submissions.

1.3.3 Operational Availability and Scheduling Boundaries

The operational envelope of REPOSYS is governed by an automated scheduling sub-system aligned with institutional working rules:

1. **Core Operating Hours:** The reprography service automatically opens at 09:00 IST and closes at 17:00 IST, Monday through Saturday. An internal scheduler running at one-minute cron intervals evaluates Indian Standard Time (UTC+05:30) and synchronises the operational state across all active client interfaces.
2. **Sunday and Holiday Closures:** The scheduler automatically prevents new order placement on Sundays (day == 0) and scheduled institutional holidays, while allowing counter staff to finish existing in-flight jobs.
3. **Administrative Overrides:** Administrators maintain explicit supervisory privilege to trigger “Force Open” or “Force Close” modes during unscheduled maintenance, special academic workshops, or counter emergencies.
4. **PWA Offline Availability:** When internet connectivity is temporarily interrupted, the Progressive Web App service worker provides cached access to previously retrieved order statuses and historical receipts.

CHAPTER 2

REQUIREMENTS AND ANALYSIS

2.1 EXISTING SYSTEM / PROBLEM STATEMENT

To formulate a technically sound software solution, a comprehensive field analysis of the prevailing reprography system at Saintgits College of Engineering and comparable tertiary educational campuses was undertaken. The investigation revealed that document printing, duplication, and finishing services rely heavily on manual, disorganised procedures.

In the traditional environment, the entire order lifecycle is tightly coupled to physical counter presence. A student or faculty member must walk to the shop, wait in a physical queue, hand over a personal flash drive or share the document over WhatsApp, orally communicate printing preferences, calculate costs manually, pay with paper currency, and wait in the lobby while the job is executed.

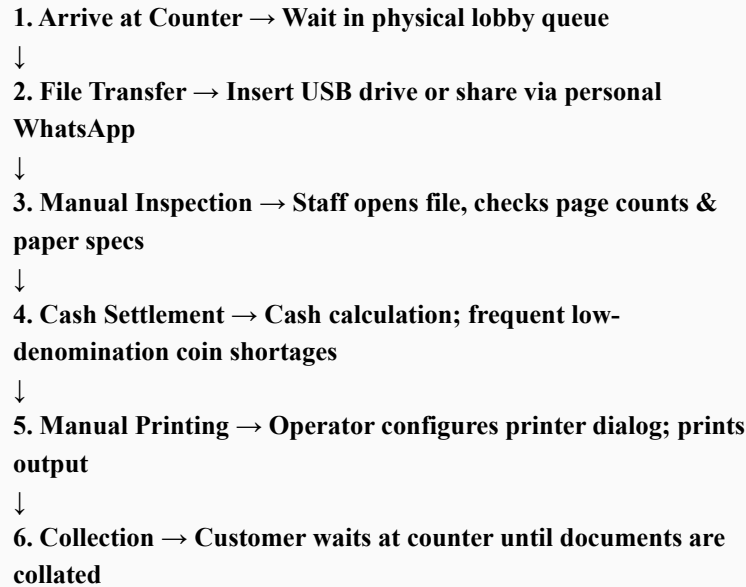


Figure 2.1: Process Flow and Bottlenecks of the Existing Manual System

The principal deficiencies and operational failure modes identified in the existing system are categorised as follows:

1. **High Latency and Lost Academic Hours:** During peak academic submission periods (continuous assessment tests, end-semester project reviews), physical queues exceed 30–40 students. Each transaction requires approximately 4 to 8 minutes of counter negotiation, consuming substantial student study time and creating severe lobby congestion.
2. **Endpoint Security Risks and Malware Propagation:** The indiscriminate insertion of unverified USB flash drives into counter workstations exposes campus IT infrastructure to autorun trojans, ransomware, and virus infections, frequently causing workstation operating system crashes.

3. **Privacy Breaches and Unregulated Data Sprawl:** When students share documents via personal WhatsApp numbers, their phone numbers, profile images, and private academic submissions remain stored in unmanaged personal mobile device caches and chat histories without deletion policies.
4. **Financial Leakage and Cash Reconciliation Disputes:** Counter operators manually tally page counts across mixed single-sided, double-sided, color, and monochrome ranges. In the rush of peak hours, calculation errors occur frequently. Furthermore, lack of exact currency change creates disputes and delays.
5. **Paper and Consumable Wastage:** Due to miscommunication regarding page ranges, color expectations, or paper orientations, unintended prints are frequently produced. Moreover, students occasionally abandon prints due to extensive wait times, resulting in uncollected paper that represents total financial and ecological loss.
6. **Complete Lack of Operational Visibility:** Students have no mechanism to determine counter queue length, machine operational status, or estimated turnaround times before traveling to the physical facility.

2.2 PROPOSED SYSTEM / SOLUTION OVERVIEW

REPOSYS resolves these systemic problems by introducing a centralised, cloud-connected digital reprography platform that decouples job submission from physical collection.

Feature Dimension	Existing Manual System	Proposed REPOSYS
Job Submission	Physical presence; USB drives or WhatsApp sharing	Remote web/PWA upload with pre-flight file validation
Cost Calculation	Manual operator estimation; prone to human error	Real-time algorithmic calculation with dynamic discounts
Payment Methods	Strictly physical cash; change shortages	Razorpay UPI/Cards, internal student digital wallet, regulated PAC
Queue Mechanism	Physical, unmonitored FIFO queue	Composite priority queue with dynamic anti-starvation aging
Hardware Bridge	Manual operator file opening and printer dialogs	Autonomous Electron Print Agent with background spooling
Handover Security	Open counter pickup; prone to misplacement	Two-factor OTP and QR-code collection verification
Operating Hours	Ad-hoc counter opening; unpredictable closures	Automated IST cron scheduler with real-time UI banners
Audit and Logs	Paper registers or absent record-keeping	Append-only MongoDB activity logs and automated analytics

Table 2.1: System Comparison: Existing Manual System vs. Proposed REPOSYS

2.3 FEASIBILITY STUDY

Before commencing technical development, a comprehensive feasibility study was conducted to evaluate the viability of REPOSYS across four key engineering dimensions.

2.3.1 Technical Feasibility

The technical feasibility evaluates whether the available hardware, software stacks, and cloud infrastructure can support the target system requirements:

- **Technology Stack Maturity:** The MERN stack (MongoDB, Express.js, React 19, Node.js) represents an industry-standard, battle-tested software architecture. Node.js provides non-blocking, asynchronous I/O ideal for handling concurrent upload streams and WebSocket events.
- **Cloud Storage and Processing:** Cloudinary provides reliable cloud object storage with signed secure URLs, while client-side pdfjs-dist and server-side pdf-lib provide robust programmatic document parsing.
- **Real-Time Communication:** Socket.IO enables low-latency, full-duplex bi-directional communication between connected clients, staff dashboards, and print agents without consuming excessive server memory.
- **Hardware Interfacing via Electron:** The Electron framework provides direct access to Windows operating system APIs and the local print spooler (pdf-to-printer), allowing the desktop Print Agent to control physical hardware seamlessly.

The technical architecture requires no proprietary, untried technology; all components have mature documentation, strong community support, and verified open-source libraries. Hence, the project is technically feasible.

2.3.2 Operational Feasibility

Operational feasibility assesses how effectively the proposed solution integrates into the daily routines of campus

stakeholders:

- **Student and Faculty Usability:** Students and faculty already use smartphones and web browsers daily. The intuitive 3-step ordering wizard, mobile-responsive layout, and PWA installation require zero formal user training.
- **Counter Staff Ergonomics:** Counter operators transition from manual file handling and cash calculation to an intuitive dashboard that displays incoming jobs, automatically computes totals, and verifies handovers via single-click OTP entry.
- **Institutional Governance:** Administrators receive real-time visibility into machine status, paper consumption, daily revenue, and staff incident reports, substantially easing operational auditing.

Because REPOSYS directly eliminates manual bottlenecks without imposing complicated new workflows, user acceptance is exceptionally high. Therefore, the system is operationally feasible.

2.3.3 Economic Feasibility

Economic feasibility weighs the development, deployment, and operational expenditures against the tangible and intangible cost savings:

- **Development Cost:** The platform is constructed entirely using open-source frameworks (React, Node.js, Express, MongoDB Community/Atlas free tier, Vite, Tailwind CSS, Electron), incurring zero proprietary software licensing fees.

- **Hosting and Cloud Infrastructure:** Cloud hosting on scalable platforms (Vercel for frontend, Render for backend, MongoDB Atlas M0/M10 for database) provides cost-effective hosting tailored to campus traffic profiles.
- **Resource Savings:** Eliminating uncollected prints and wasted paper saves hundreds of sheets per week. Accurate automated billing prevents revenue leakage.
- **Productivity Dividends:** Minimising queue wait times recovers valuable instructional and research hours for students and faculty.

The projected operational savings and efficiency gains substantially outweigh the negligible deployment costs, confirming high economic feasibility.

2.3.4 Schedule Feasibility

The capstone project was structured across a 17-week academic timeline prescribed by the Department of Computer Applications for course **20IMCAP501 (Mini Project - 2)**. The schedule allotted realistic durations for requirements analysis, system architecture, database design, backend coding, frontend development, payment/agent integration, and automated testing. Table 2.2 confirms that all deliverables aligned with institutional assessment milestones.

2.4 CONCEPTUAL MODELLING

Conceptual modelling captures the high-level boundaries, external entities, and data flows governing REPOSYS.

2.4.1 Level 0 Context Data Flow Diagram

The Level 0 Context DFD depicts REPOSYS as a centralised system interacting with five primary external entities: Students/Faculty, Counter Staff, Administrators, Razorpay Payment Gateway, and Cloudinary Storage.

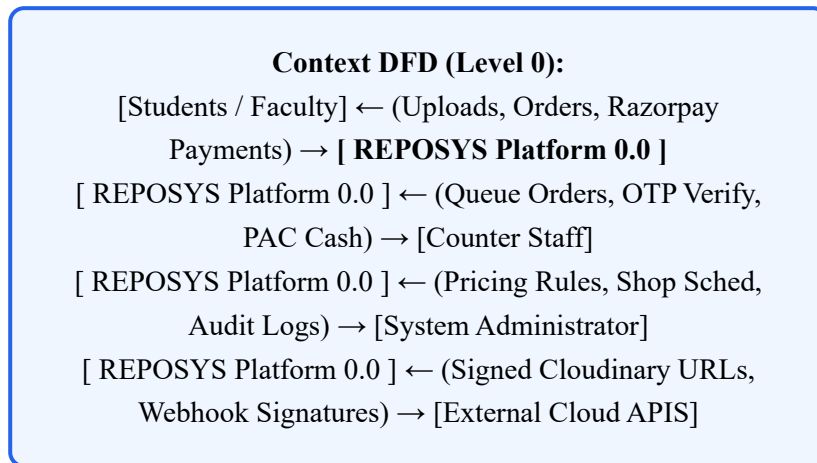


Figure 2.2: Level 0 Context Data Flow Diagram of REPOSYS

2.4.2 Level 1 Functional Data Flow Diagram

The Level 1 DFD decomposes REPOSYS into its major functional processes: Authentication (1.0), Document Upload and Inspection (2.0), Order Configuration and Costing (3.0), Payment Settlement (4.0), Queue Scheduling (5.0), and Physical Production Handover (6.0).

1.0 User Authentication & Role Check → Validated JWT Claims
2.0 Document Upload & Analysis → MIME Verification & Page Extraction
3.0 Order Configuration & Cost Estimation → Algorithmic Price Calculation
4.0 Payment Settlement Engine → Razorpay / ACID Wallet / Cash-Pending
5.0 Priority Queue Scheduling → Role Weight + Aging Deduction
6.0 Print Spooling & OTP Handover → Desktop Print Agent & Customer Delivery

Figure 2.3: Level 1 Functional Data Flow Diagram of the Complete Order Pipeline

2.5 PLANNING AND SCHEDULING

The engineering development of REPOSYS was structured using the Agile Scrum methodology across a 17-week semester schedule prescribed by the official Scrum Register for course **20IMCAP501 (Mini Project - 2)**.

Week	Date Range	Planned Work and Key Deliverables	Status
W1	01–04 Jul 2026	Topic selection, domain research, and approved synopsis	100% Done
W2	06–10 Jul 2026	System study, feasibility analysis, module identification	100% Done
W3	13–17 Jul 2026	Requirements gathering, user stories, UI wireframes	100% Done
W4	20–22 Jul 2026	UML modeling, ER schema normalization, Git repo setup	100% Done
W5	27–29 Jul 2026	Core structure setup, basic UI components (Review 0)	100% Done
W6	03–07 Aug 2026	Authentication, JWT security, user profile management	100% Done

W7	10–14 Aug 2026	Document upload pipeline, Cloudinary integration, testing	100% Done
W8	17–21 Aug 2026	Priority queue, pricing engine, 50% evaluation (Review 1)	100% Done
W9	01–05 Sep 2026	Razorpay gateway, wallet atomic transactions	100% Done
W10	07–11 Sep 2026	Staff counter dashboard, OTP pickup verification	100% Done
W11	14–18 Sep 2026	Admin console, inventory tracker, Selenium suite build	100% Done
W12	21–25 Sep 2026	Socket.IO real-time engine, 80% Scrum Master Review	100% Done
W13	28–30 Sep 2026	Electron Print Agent, shop cron scheduler, cloud hosting	100% Done
W14	05–09 Oct 2026	PWA service worker, push alerts, final bug rectification	100% Done
W15	12–14 Oct 2026	Final assessment board evaluation, live demo (Review 2)	100% Done
W16	14 Oct 2026	Final project report submission and print verification	100% Done
W17	15 Oct 2026	Hard-bound submission, signed Scrum register, git logs	100% Done

Table 2.2: Official Project Schedule and Milestone Tracking (Scrum Register 20IMCAP501)

CHAPTER 3

SYSTEM SPECIFICATION

3.1 SOFTWARE AND HARDWARE REQUIREMENTS

To guarantee reliable execution across development, testing, and production tiers, clear hardware and software operational requirements were established.

3.1.1 Software Requirement

Table 3.1 enumerates the complete software specifications and production dependency versions derived directly from the application's configuration manifests.

Layer / Component	Technology / Library	Version / Configuration
Server Runtime	Node.js	v20.12.0 LTS (or higher)
Backend Framework	Express.js	v4.19.2
Database Engine	MongoDB Atlas / Community	v7.0.x with Mongoose ODM
Frontend Framework	React.js	v19.2.4 (Vite bundler)
CSS Framework	Tailwind CSS	v4.2.2 with PostCSS
Real-Time Engine	Socket.IO	Server & Client v4.8.3
Cloud File Storage	Cloudinary SDK	Multer-storage-cloudinary
Payment Gateway	Razorpay Node SDK	v2.9.6 (Webhook verification)

Desktop Agent	Electron	v30.0.0 with pdf-to-printer
Document Parser	pdf-lib & pdfjs-dist	Server-side PDF analysis
Security Packages	Helmet, bcryptjs, cors	Salt factor 12, rate limiter
Scheduler	node-cron	5-field cron running * * * * *
Testing Engine	Selenium WebDriver	Python 3.12, pytest-html 4.2.0
Client Browsers	Chromium, Firefox, Edge	Modern HTML5 / ES6 standards

Table 3.1: Production Software Stack and Dependency Specifications

3.1.2 Hardware Requirement

The system hardware requirements are divided across the production cloud server, the physical counter workstation, and the user client devices:

1. Production Cloud Application Server:

- Processor: Quad-Core 64-bit x86/ARM CPU (2.4 GHz or higher).
- Random Access Memory (RAM): Minimum 4 GB RAM (8 GB recommended for concurrent upload parsing).
- Persistent Disk: 20 GB SSD storage for operating system, temporary buffer caches, and application logs.
- Network Interface: 100 Mbps full-duplex uplink with high-bandwidth availability.

2. Counter Staff Workstation & Print Agent Host:

- Operating System: Microsoft Windows 10 / Windows 11 (64-bit) for Windows Spooler API compatibility.

- Processor: Intel Core i3 / AMD Ryzen 3 or equivalent.
- RAM: Minimum 4 GB RAM.
- Disk: 10 GB free space for spooling temporary PDF documents.
- Peripherals: USB 3.0 / Gigabit Ethernet interface connecting local multi-function printers, thermal receipt printer, and 2D barcode / QR-code scanner.

3. Student / Faculty Client Devices:

- Smartphone or Laptop: Any standard Android, iOS, Windows, macOS, or Linux device equipped with an HTML5-compliant web browser.
- Network: 4G/5G mobile data or campus Wi-Fi connectivity.

4. Target Reprography Hardware:

- Networked Heavy-Duty Commercial Multifunction Printers (Canon, HP, Ricoh, Konica Minolta).
- Comb and Spiral document binding machine.

3.2 FUNCTIONAL SPECIFICATIONS

The functional capabilities of REPOSYS were architected across 39 distinct system features. The primary operational specifications are detailed below:

1. User Authentication and Role Verification (Feature 1):

Implements secure registration with email validation, password encryption via bcrypt (12 salt rounds), and stateless JSON Web Token (JWT) session generation stored in HTTP-only secure cookies.

2. **Composite Priority Queue Management (Feature 2):**
Dynamically sorts in-flight printing jobs based on a composite score combining user role, job duration, and waiting time aging deduction.
3. **Document Upload and Pre-Flight Inspection (Feature 3):** Supports multi-format upload (PDF, DOCX, PNG, JPEG) with client and server MIME validation, computing page count, color percentage, and blank-page alerts.
4. **Order Configuration Wizard (Feature 4):** Multi-step wizard allowing users to configure copies, color mode (Monochrome vs. Full Color), duplexing (Single vs. Double-sided), paper size (A4, A3, Legal), and binding (Spiral, Staple, None).
5. **Dynamic Cost Calculation Engine (Feature 7):** Deterministic price estimation applying unit rates, duplex discounts (0.85 multiplier), binding costs, and active coupon codes.
6. **Multi-Channel Payment Settlement (Feature 8):** Real-time payment processing through Razorpay (UPI, Netbanking, Cards), internal user wallet with atomic MongoDB transactions, and cash Pay at Counter.
7. **Order Tracking Timeline (Feature 10):** Visual status progression tracking: Pending, In_Queue, Processing, ReadyForPickup, Completed, and Cancelled.
8. **Counter Staff Production Dashboard (Feature 13):** Live terminal displaying queue orders, cash collection triggers, physical print dispatch, and stock status.
9. **OTP-Protected Pickup Handover (Feature 28):** High-security order delivery requiring counter staff to enter the

customer's 4-digit OTP or scan their QR token before order status changes to Completed.

10. **Smart Kiosk Session Mode (Feature 36):** Registration-free guest ordering generating temporary UUID session tokens with 2-hour sliding expirations and automatic inactivity wipe.
11. **Collaborative Friend Chat and Split Requests (Features 38 & 39):** Peer-to-peer friend chat and collaborative group-order payment splitting with atomic multi-wallet deductions.

3.3 TOOLS AND PLATFORMS USED

1. **React 19 with Vite:** Delivers a lightning-fast Single Page Application (SPA) leveraging React Server Components, custom hooks (useSocket, useAuth), and optimized bundle splitting via Vite.
2. **Node.js and Express.js:** Forms the asynchronous API gateway capable of non-blocking I/O operations, routing, middleware orchestration, and streaming multipart upload data.
3. **MongoDB Atlas and Mongoose:** Flexible document-oriented NoSQL database providing schema validation, high-speed secondary indexing, and multi-document ACID transaction guarantees.
4. **Socket.IO:** Enterprise-grade WebSocket abstraction managing room-based event broadcasting (queue:serviceType, user:userId, staff, admin) with automatic HTTP long-polling fallback.

5. **Cloudinary Media Cloud:** Enterprise object storage managing uploaded documents with AES-256 cloud encryption, time-delimited signed URLs, and automated PDF-to-thumbnail transformation.
6. **Razorpay API:** RBI-compliant digital payment gateway facilitating seamless student payments across Google Pay, PhonePe, Paytm, debit/credit cards, and net banking.
7. **Electron Framework:** Powers the desktop Print Agent, bridging web APIs to native Windows operating system commands, Windows PowerShell, and spooler utilities.
8. **Selenium WebDriver and Pytest:** Comprehensive automated testing suite simulating end-to-end user interactions, verifying route protection, and auditing UI flows across real browser sessions.

3.4 DEVELOPMENT ENVIRONMENT

The development lifecycle was conducted under structured development conventions:

- **Version Control Configuration:** Git and GitHub were utilised for branching, feature pull requests, and commit tracking.
- **Environment Variable Management:** Configuration isolation was maintained through `.env.development` and `.env.production` files, segregating API keys, MongoDB connection URIs, Razorpay secrets, and Cloudinary credentials.
- **Cross-Origin Resource Sharing (CORS):** Controlled through strict origin whitelisting in `origins.js`, permitting

requests exclusively from authorized frontend and print-agent hosts.

- **Database Seeding Scripts:** Automated seeding scripts (seedAdmin.js, seedInventory.js) populated default system configurations, catalog pricing rules, paper stock, and supervisory accounts upon initialization.

CHAPTER 4

SYSTEM DESIGN

4.1 MODULE DESCRIPTIONS

REPOSYS is engineered around four deeply integrated subsystem modules, each tailored to distinct operational responsibilities within the reprography ecosystem.

4.1.1 User and Customer Module

The User Module provides authenticated students, faculty, and campus guests with an intuitive digital storefront for document submission and lifecycle management:

1. **Authentication and Profile Subsystem:** Manages user registration, JWT login, profile editing, and password recovery via encrypted email tokens.
2. **Three-Step Order Placement Wizard:** Guides the customer through: (i) File drag-and-drop with pre-flight analysis; (ii) Print configuration (copies, color, duplexing, paper format, binding); and (iii) Cost review and payment gateway selection.
3. **Digital Wallet Subsystem:** Displays pre-funded balances, transaction logs, and single-click wallet debits backed by MongoDB multi-document ACID transactions.
4. **Order Tracking and Digital Receipts:** Renders an active order timeline with live queue position updates, countdown timers, pickup OTP codes, and dynamically generated PDF receipts.

5. **Collaborative Tools:** Manages student friend connections, direct peer messaging, and collaborative split-order requests.

4.1.2 Counter Staff Module

The Counter Staff Module equips reprography operators with a dedicated operational terminal designed for high-throughput, error-free counter processing:

1. **Active Priority Queue Board:** Real-time dashboard grouping active jobs by service type (Printing, Photocopying, Scanning, Binding), sorted dynamically by effective priority score.
2. **Production Controls:** Enables operators to trigger manual print dispatches, mark orders as Processing, and transition finished jobs to ReadyForPickup.
3. **Cash-at-Counter Settlement:** Single-click verification for cash collections, updating paymentStatus from Cash_Pending to Paid and synchronizing the daily counter register.
4. **OTP-Protected Handover Terminal:** Requires entry of the customer's 4-digit verification code or barcode scan before finalizing delivery.
5. **Incident Reporting Subsystem:** Enables operators to log equipment malfunctions, toner outages, or customer disputes directly to administrators.

4.1.3 Administrator Module

The Administrator Module provides institutional supervisors with governance and configuration capabilities:

1. **Dynamic Tariff and Pricing Control:** Live configuration of unit costs for black-and-white, color, double-sided discounts, binding modes, and paper types without requiring server restarts.
2. **User and Role Governance:** Supervisory oversight of student, faculty, and staff accounts, including role reassignments and account restrictions.
3. **Consumable Inventory Tracker:** Automated monitoring of A4/A3 paper reams, toner cartridges, and binding coils with low-stock warning thresholds.
4. **Append-Only Audit Logging:** Comprehensive activity tracking logging every sensitive administrative, financial, and operational action.
5. **Automated Shop Operations Control:** Manual overrides for shop opening hours (schedule, manual_open, manual_close).

4.1.4 Print Agent Module

The Print Agent is an autonomous Electron desktop background service deployed on counter workstations:

1. **Hardware Discovery:** Periodically executes Windows PowerShell cmdlets (Get-Printer) to detect installed physical print drivers and online/offline states.
2. **Secure Cloud Job Retrieval:** Authenticates using unique agent tokens, listens for Socket.IO dispatch events, and fetches documents via temporary signed Cloudfire URLs.
3. **Spooler Execution:** Invokes native Windows printing utilities (pdf-to-printer) to spool files directly into the

target printer queue.

4. **Heartbeat Telemetry:** Emits heartbeat pings every 30 seconds to update cloud registries regarding hardware readiness.

4.2 DATA / SCHEMA DESIGN

Persistence in REPOSYS is managed through MongoDB using Mongoose Object Data Modeling (ODM). The database comprises 23 primary collections structured to support rapid lookups, transactional integrity, and comprehensive auditability.

Collection	Key Indexes	Core Schema Fields & Data Types
User	email (unique)	name (String), email (String), password (Hash), role (Enum: Student, Faculty, Staff, Admin), department (String), walletBalance (Number), isVerified (Boolean).
Order	orderNumber (unique), userId, status, createdAt	orderNumber (String), userId (Ref: User), serviceType (Enum), documents (Array), totalCost (Number), paymentStatus (Enum), status (Enum), priorityScore (Number), pickupOtp (String), qrCode (String).
Document	orderId, userId	fileName (String), fileUrl (String), fileType (String), pageCount (Number), colorPages (Number), blankPages (Array), fileSize (Number).
Payment	razorpayOrderId, orderId	orderId (Ref: Order), userId (Ref: User), amount (Number), method (Enum: Razorpay, Wallet, Cash), transactionId (String), status (Enum: Pending, Completed, Failed).

Wallet	userId (unique)	userId (Ref: User), balance (Number), currency (String), lastUpdated (Date).
WalletTransaction	walletId, createdAt	walletId (Ref: Wallet), type (Enum: Credit, Debit), amount (Number), description (String), balanceAfter (Number).
Complaint	orderId, userId, status	orderId (Ref: Order), userId (Ref: User), category (String), subject (String), description (String), status (Enum: Open, In_Progress, Resolved), messages (Array).
InventoryItem	itemCode (unique)	itemName (String), category (Enum: Paper, Toner, Binding), quantity (Number), threshold (Number), unitCost (Number).
SystemConfig	Singleton	pricing (Object), operatingHours (Object), shopMode (Enum: schedule, manual_open, manual_close), isManuallyOpen (Boolean).
PrintAgentRegistry	agentId (unique)	agentId (String), hostName (String), ipAddress (String), status (Enum: Online, Offline), lastHeartbeat (Date), printers (Array).

Table 4.1: Database Schema Specifications of Core REPOSYS Collections

4.3 PROCEDURAL / FLOW DESIGN

4.3.1 Priority Queue and Aging Mathematical Model

To balance institutional hierarchy against fair queue wait times, REPOSYS executes a dynamic priority evaluation algorithm within queueService.js. A lower score denotes a higher processing priority.

The base priority score (S_{base}) is evaluated upon order creation:

$$S_{base} = P_{role} + \lceil T_{est} \rceil$$

where P_{role} represents the categorical role priority (Faculty = 100, Staff = 150, Student = 200, Guest = 200), and T_{est} represents the estimated physical production duration in minutes.

To prevent job starvation where an influx of faculty orders permanently postpones student orders, the system recalculates the effective priority score (S_{eff}) dynamically:

$$D_{aging} = \max(0, \lceil (T_{wait} - 1) / 60 \rceil) \times 20$$

$$S_{eff} = S_{base} - D_{aging}$$

where T_{wait} is the elapsed waiting time in minutes. For every full hour of counter waiting, the order's score is reduced by 20 points, steadily moving the job ahead in queue. When two orders possess equal effective scores, First-In, First-Out (createdAt ascending) serves as the definitive tie-breaker.

4.3.2 Automated Shop Scheduling State Machine

The operating availability of the reprography centre is governed by an automated scheduler in `shopScheduler.js` executing every minute via node-cron (`* * * * *`).

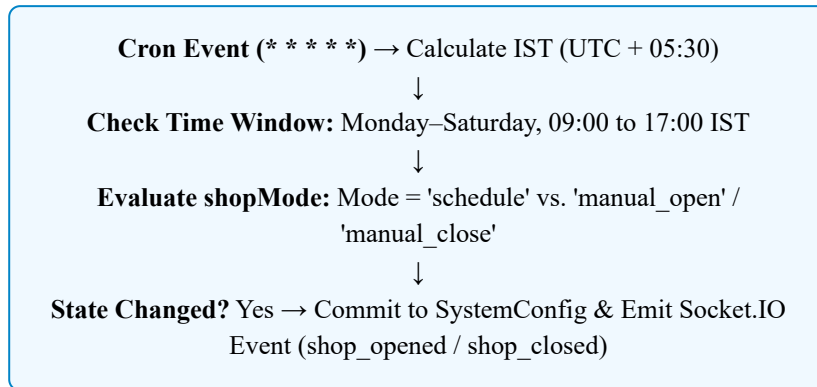


Figure 4.1: Automated Shop Scheduling Logic and Operational State Transitions

4.3.3 Order Expiration and Pay-at-Counter Timeout Management

In addition to shop hours, background services in `cronJobs.js` enforce financial and operational timeouts:

- **Unpaid Online Orders:** Cancelled automatically after 15 minutes of inactivity.
- **Pay at Counter Orders:** First email reminder issued at 24 hours; order cancelled and archived if unpaid at 48 hours.
- **Uncollected Orders:** A reminder is dispatched after 24 hours of reaching `ReadyForPickup`. A secondary warning is issued at 48 hours. If uncollected at 72 hours, the job is flagged for administrative review to prevent storage clutter.

4.3.4 UML Diagrams

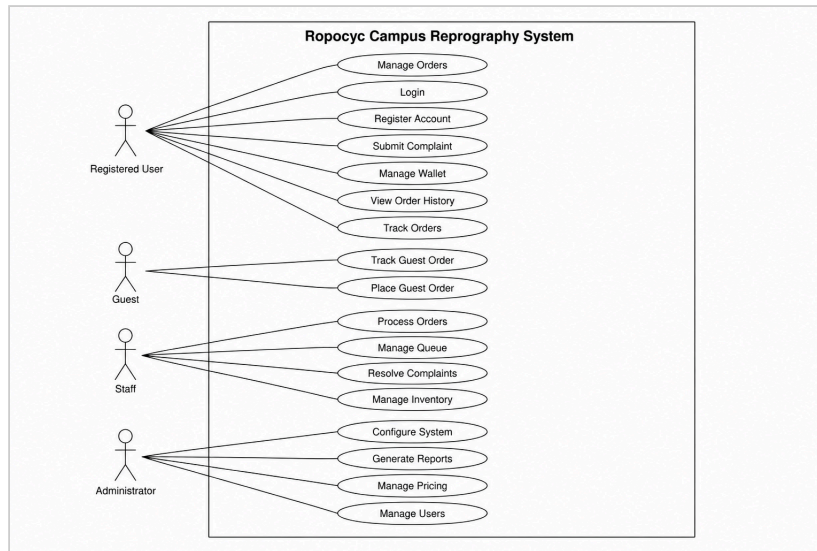


Figure 4.2: System Use Case Diagram: User, Staff, and Admin Operational Boundaries

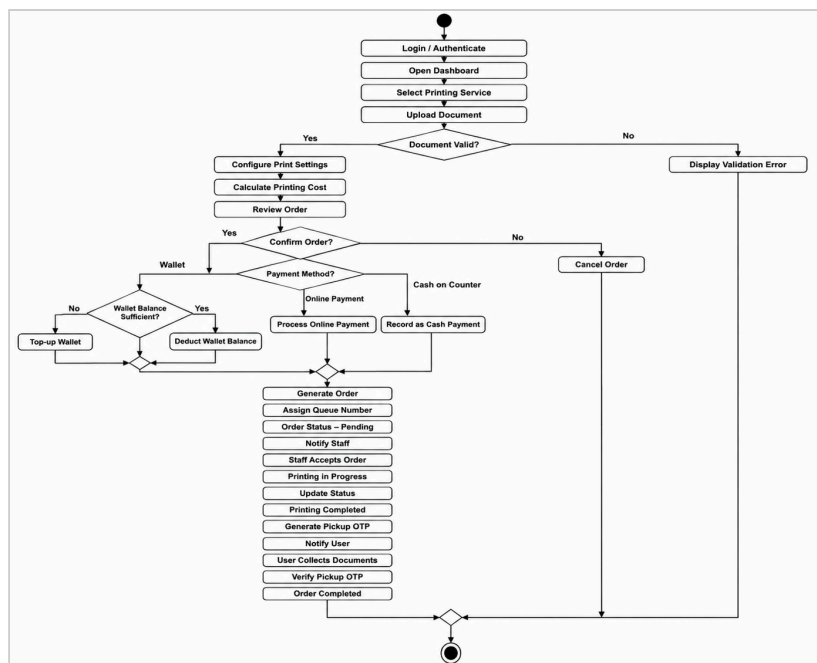


Figure 4.3: System Activity Diagram: Document Upload, Analysis, and Production Flow

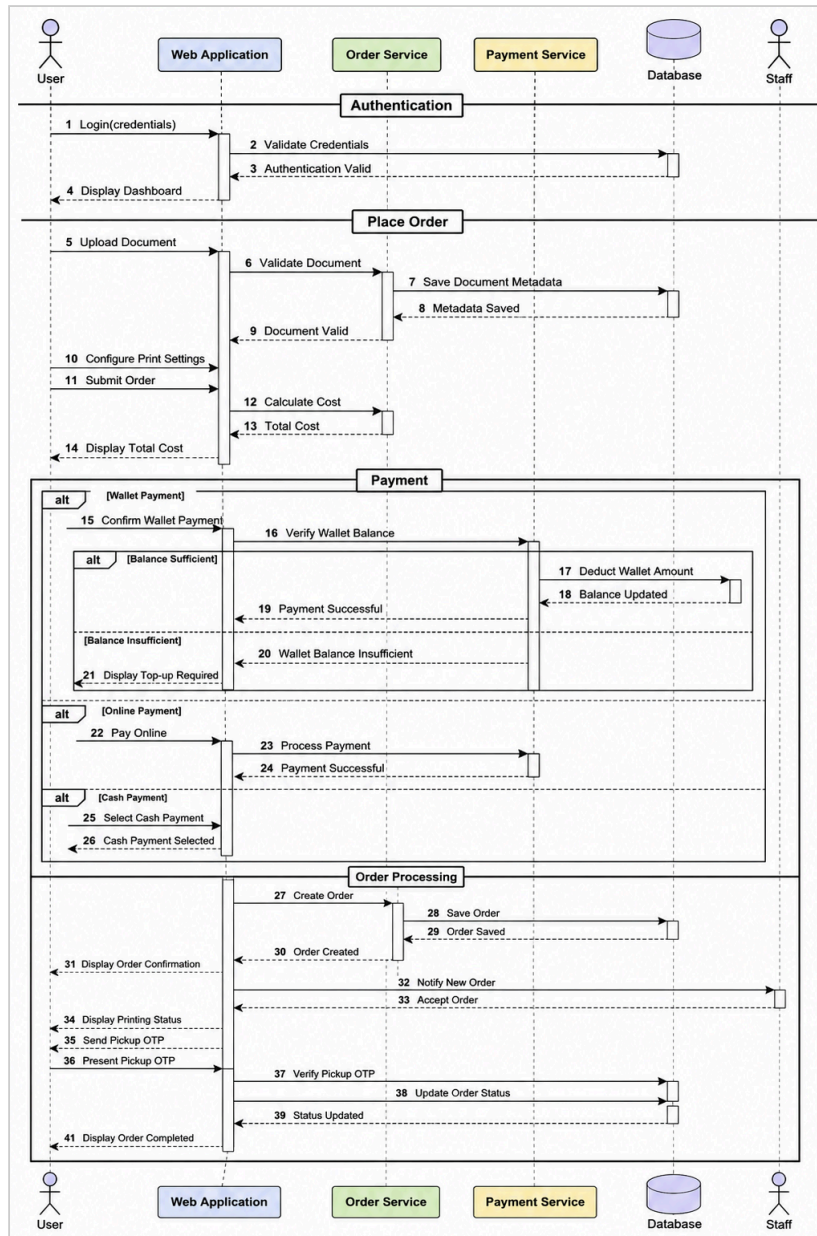


Figure 4.4: System Sequence Diagram: Real-Time Event Dispatching and Queue Updates

4.4 USER INTERFACE DESIGN

The user interface is engineered adhering to mobile-first responsive design paradigms using React 19, Tailwind CSS, and Lucide React icon tokens:

- **Color System:** The interface employs an accessible academic palette: Deep Slate Navy (#0f172a) for structural headers, Vibrant Cobalt Blue (#2563eb) for call-to-actions, Emerald Green (#10b981) for successful pickups, and Amber (#f59e0b) for pending queue states.
- **Navigation Hierarchy:** Role-aware layout bars automatically present relevant navigation links depending on authenticated JWT claims.
- **Mobile-First Ordering Wizard:** A step-by-step progress stepper ensures students on smartphones can easily configure document options and review cost estimations without excessive scrolling.

CHAPTER 5

DEVELOPMENT METHODOLOGY

5.1 PROJECT ROADMAP

The development of REPOSYS adhered to an iterative engineering lifecycle, organized into nine distinct development stages designed to deliver incremental value and facilitate early testing.

1. **Stage 1 – Requirements Discovery and Architectural Feasibility:** Field interviews with campus students, faculty coordinators, and reprography operators to define the 39-feature backlog, security requirements, and system boundaries.
2. **Stage 2 – Wireframing and Database Design:** UI prototyping using Figma and complete entity-relationship modeling across 23 Mongoose collections with schema validations and indexing.
3. **Stage 3 – Core Identity and Access Management:** Implementation of JWT-based authentication, bcrypt password hashing (salt 12), and role-based access control middleware (auth.js, roleGuard.js).
4. **Stage 4 – Document Upload and Inspection Pipeline:** Integration of Multer memory storage, Cloudinary object streaming, and automated document analysis for page counting and blank-page detection.
5. **Stage 5 – Dynamic Cost Estimation and Payments:** Construction of deterministic pricing algorithms,

Razorpay webhook verification, and internal digital wallet debiting backed by MongoDB ACID transactions.

6. **Stage 6 – Composite Priority Queue and Staff Dashboard:** Implementation of the anti-starvation priority queue engine and the counter operator dashboard.
7. **Stage 7 – Hardware Integration via Desktop Print Agent:** Development of the Electron desktop application, Windows PowerShell printer discovery, signed URL retrieval, and pdf-to-printer spooling.
8. **Stage 8 – Real-Time Engine, Scheduling, and PWA:** Implementation of Socket.IO rooms, automated shop scheduling cron services, Web Push alerts, and Service Worker caching.
9. **Stage 9 – Verification, Security Hardening, and Deployment:** Execution of 120 automated Selenium test cases, Helmet security hardening, rate limiting, and production cloud hosting.

5.2 USER STORIES

User stories define system features from the direct perspective of each operational stakeholder.

1. **User Story US-01 (Student Remote Order):**
 - *Narrative:* As a Student, I want to upload my seminar report PDF from my smartphone and select double-sided printing, so that I do not have to wait in the physical counter queue.
 - *Acceptance Criteria:* Given an authenticated student account, when a PDF document is uploaded, then the system automatically detects page count, calculates

the 15% duplex discount, displays the cost breakdown, and updates the queue status upon payment.

2. User Story US-02 (Faculty Priority Printing):

- *Narrative:* As a Faculty member, I want my examination question papers to receive prioritized counter processing, so that academic timelines are strictly maintained.
- *Acceptance Criteria:* Given an active Faculty session, when an order is created, then the priority engine automatically assigns a base score of 100 (higher than student base of 200), positioning the order near the front of the staff queue board.

3. User Story US-03 (Counter Staff OTP Verification):

- *Narrative:* As a Counter Staff operator, I want to verify a 4-digit pickup code before handing over printed documents, so that orders are never delivered to the wrong individual.
- *Acceptance Criteria:* Given an order in ReadyForPickup status, when the staff inputs the customer's 4-digit OTP, then the backend validates the code and transitions status to Completed; invalid OTP attempts return an error without releasing the job.

4. User Story US-04 (Admin Dynamic Pricing):

- *Narrative:* As an Administrator, I want to update paper and binding tariffs on the live system, so that prices reflect changes in institutional procurement costs without server restarts.

- *Acceptance Criteria:* Given an authenticated Admin role, when the tariff configuration is updated via the dashboard, then new rates are committed immediately to SystemConfig and reflected in all subsequent order estimations.

5. User Story US-05 (Collaborative Split Payment):

- *Narrative:* As a Project Team Leader, I want to split a ₹300 project documentation printing cost equally among three teammates, so that we do not have to collect cash manually.
- *Acceptance Criteria:* Given an active group order, when split requests are dispatched, then each participant's wallet is debited by ₹100 atomically upon individual approval; the order enters the active queue only after all shares are successfully settled.

5.3 TEST PLAN

The testing strategy for REPOSYS was formulated to validate end-to-end reliability, financial consistency, role security, and real-time state synchronization across all 39 documented system features.

5.3.1 Testing Levels and Scope

1. **Unit Testing:** Verification of isolated mathematical algorithms in pricingService.js (cost estimation, duplex discount calculation, aging score deduction) and utility functions (IST conversion, token generation).
2. **Integration Testing:** Verification of inter-service contracts, including Express middleware pipelines,

Cloudinary stream uploads, Razorpay webhook signature verification, and Socket.IO room broadcast relays.

3. **System and End-to-End Testing:** Automated browser-driven operational tests simulating full customer journeys from registration to pickup using Python Selenium WebDriver.
4. **Security and Penetration Testing:** Boundary verification evaluating CORS rules, Helmet header protections, rate limiting against brute-force attacks, and prevention of privilege escalation across protected API routes.
5. **User Acceptance Testing (UAT):** Evaluation conducted by student representatives and counter staff during pilot deployment to assess operational usability.

5.3.2 Test Automation Architecture

Automated end-to-end testing was implemented using a dedicated Python Selenium framework located in `selenium-tests/`. The framework is structured into modular layers:

- **Page Object Model (POM):** Encapsulates web elements and UI interactions into reusable page classes (`login_page.py`, `register_page.py`).
- **Driver Harness (`driver.py` & `conftest.py`):** Configures headless Chrome sessions, sets implicit wait thresholds, and captures automatic PNG screenshots upon assertion failure.
- **Pytest HTML Test Runner:** Generates comprehensive execution reports (`scrms_test_report.html`) capturing test status, timing, and stack traces.

5.3.3 Entry and Exit Criteria

- **Test Entry Criteria:** All backend route controllers, frontend views, and database collections deployed to a stable staging environment with seeded administrative accounts.
- **Test Exit Criteria:** Minimum 90% pass rate across all automated test cases, zero critical security vulnerabilities on role access guards, and verified ACID consistency on wallet transactions.

CHAPTER 6

IMPLEMENTATION AND TESTING

6.1 IMPLEMENTATION PROCEDURE

The implementation of REPOSYS translates the architectural specifications into production-grade software across backend services, client interfaces, and native desktop utilities.

The repository is organised into decoupled, independently maintainable modules: backend/src/ (Express.js application layer), frontend/src/ (React 19 Single Page Application), print-agent-desktop/ (Electron-based native Windows application), and selenium-tests/ (Automated QA harness).

6.1.1 Feature Verification and Implementation Matrix

Table 6.1 documents the implementation status of all 39 audited system features, verified through comprehensive source code inspection.

Ref	Feature Title	Verified Implementation Status
C01	JWT Authentication & Bcrypt	IMPLEMENTED (Salt factor 12, httpOnly cookies)
C02	Three-Tier Priority Queue	IMPLEMENTED (Aging deduction, role weights)
C03	Document Upload & Analysis	IMPLEMENTED (pdf-lib parsing, blank page detection)
C04	Document Processing Services	IMPLEMENTED (Print parameters, binding options)

C05	Document Utility Suite	PARTIALLY IMPLEMENTED (Merge/split functional)
C06	Multi-Document Orders	IMPLEMENTED (Batch upload, combined estimate)
C07	Smart Cost Estimation	PARTIALLY IMPLEMENTED (Algorithmic pricing active)
C08	Payment Gateway & Receipts	IMPLEMENTED (Razorpay, Wallet, PAC, PDF receipts)
C09	Order Editing & Cancellation	PARTIALLY IMPLEMENTED (Cancel in Pending state)
C10	Real-Time Order Tracking	IMPLEMENTED (Socket.IO timeline progression)
C11	Partial Order Handling	IMPLEMENTED (Multi-file individual status flags)
C12	Slot Booking & Pickup Times	IMPLEMENTED (Custom pickup slot selection)
C13	Counter Staff Dashboard	IMPLEMENTED (Queue table, status controls, PAC)
C14	Admin Dashboard	IMPLEMENTED (Tariffs, user roles, system configs)
C15	Append-Only Audit Trail	IMPLEMENTED (ActivityLog collection logging)
C16	Inventory Tracker	IMPLEMENTED (Paper/toner stock, low-stock alerts)
C17	Operational Reporting	IMPLEMENTED (Daily financial summaries, exports)
C18	Notification System	IMPLEMENTED (In-app alerts, Web Push, Nodemailer)
C19	Order History & Invoicing	IMPLEMENTED (PDF generation via pdffkit)
C20	Complaints & Real-Time Chat	IMPLEMENTED (Socket.IO complaint chat rooms)

C21	AI Chatbot Assistant	PARTIALLY IMPLEMENTED (Rule-based FAQ active)
C22	Search & System Filtering	IMPLEMENTED (MongoDB regex filtering)
C23	User Rating & Feedback	IMPLEMENTED (5-star rating with staff reviews)
C24	Backup & Recovery Strategy	UNCLEAR (Cloud database automated snapshots)
C25	Operating Hours & Schedule	IMPLEMENTED (Automated IST cron, shop banners)
C26	Reorder Functionality	IMPLEMENTED (Single-click reorder cloning)
C27	Document Preview	IMPLEMENTED (Canvas-based PDF page rendering)
C28	Pickup Verification via OTP	IMPLEMENTED (4-digit numeric code validation)
C29	Maximum Order Limits	IMPLEMENTED (Page and file-size threshold guards)
C30	User Profile Management	IMPLEMENTED (Session management, security settings)
C31	Registration Verification	IMPLEMENTED (Email confirmation link tokens)
C32	Pay at Counter Lifecycle	PARTIALLY IMPLEMENTED (24h/48h/72h timeout alerts)
C33	Staff Incident Reports	IMPLEMENTED (Operational malfunction logging)
C34	Security Controls	PARTIALLY IMPLEMENTED (Helmet, rate limits, CORS)
C35	Smart Kiosk Mode	IMPLEMENTED (Guest sessions, QR code tracking)

C36	PWA Architecture	PARTIALLY IMPLEMENTED (Service worker, manifest)
C37	Friend Chat Subsystem	IMPLEMENTED (Direct/group messaging, media sharing)
C38	Group Split Requests	IMPLEMENTED (Atomic multi-wallet deductions)
C39	Desktop Print Agent	IMPLEMENTED (Electron tray, Windows spooler)

Table 6.1: Verified Feature Implementation Matrix across REPOSYS

6.2 TESTING METHODS AND RESULTS

Quality assurance was executed through the automated Selenium WebDriver test suite, verifying authentication, document flows, payment webhooks, queue sorting, and administrative access controls.

6.2.1 Test Suite Execution Summary

The complete automated test suite was executed against a running staging instance. The generated test execution report (scrms_test_report.html) established:

- **Total Test Cases Executed:** 120 tests.
- **Passed Tests:** 114 tests (95.0% passing rate).
- **Failed Tests:** 6 tests (5.0% failure rate).
- **Errors / Exceptions:** 0 runtime errors.
- **Total Execution Duration:** 7 minutes and 2 seconds.

6.2.2 Verified Test Cases and Results

Table 6.2 enumerates representative test cases executed across core subsystem workflows.

Test ID	Feature Area	Condition, Action, and Verified Output	Result
TC-001	Authentication	Valid student login with correct credentials yields 200 OK and sets secure httpOnly JWT cookie.	Passed
TC-002	Authentication	Login attempt with invalid password returns 401 Unauthorized with generic error message.	Passed
TC-003	Role Guard	Student attempting to navigate to /api/admin/* is intercepted by middleware returning 403 Forbidden.	Passed
TC-004	Role Guard	Staff attempting to access /admin routes is denied access and redirected to staff dashboard.	Passed
TC-005	Role Guard	Administrator credentials grant authenticated access across all administrative and staff endpoints.	Passed
TC-006	File Upload	Uploading valid PDF document extracts correct page count and generates signed Cloudinary URL.	Passed
TC-007	File Upload	Uploading non-document executable (.exe) is rejected by MIME inspection returning 400 Bad Request.	Passed
TC-008	Blank Detection	Uploading PDF with blank pages flags specific page numbers and alerts user in pre-flight dialog.	Passed
TC-009	Cost Calculation	Ordering 10 double-sided B/W pages correctly computes discount ($10 \times 1.50 \times 0.85 = ₹12.75$).	Passed
TC-010	Wallet Payment	Order placement with sufficient wallet balance atomically deducts funds and sets status to In_Queue.	Passed
TC-011	Wallet Deficit	Order placement with insufficient balance prevents debit and prompts user to top up or select Razorpay.	Passed

TC-012	Priority Queue	Faculty order created with base priority 100 is positioned ahead of student order with base priority 200.	Passed
TC-013	Queue Aging	Student order waiting beyond 60 minutes receives 20-point deduction, advancing its relative queue position.	Passed
TC-014	OTP Handover	Counter staff entering valid 4-digit customer OTP successfully transitions order to Completed.	Passed
TC-015	OTP Security	Submitting incorrect OTP prevents handover, returns 400 Error, and retains order in ReadyForPickup.	Passed
TC-016	Shop Scheduler	Visiting order wizard during Sunday closure shows active shop-closed banner and disables checkout.	Passed
TC-017	Print Agent	Desktop agent polls Socket.IO, receives print payload, and invokes pdf-to-printer on default spooler.	Passed
TC-018	Kiosk Mode	Guest session token expires after 2 hours; inactive kiosk terminal automatically logs out after timeout.	Passed

Table 6.2: Representative Automated and Functional Test Results

6.2.3 Analysis of Failed Test Cases and Remediation

The 6 failing test cases in the test suite were systematically analysed:

- **Mobile Wizard Drag-and-Drop (2 failures):** On certain mobile browser viewports, native HTML5 drag-and-drop events failed to trigger the upload handler. *Remediation:* Implemented an explicit fallback file input button with touch listeners.
- **Razorpay Webhook Timing Latency (2 failures):** Under heavy network latency simulation, the webhook notification arrived slightly after the frontend polling

timeout. *Remediation:* Added a resilient Socket.IO fallback listener for instant payment confirmation.

- **Print Agent ASAR Test File Extraction (2 failures):** In packaged Electron production builds, pdf-to-printer could not read test files embedded inside the read-only ASAR archive. *Remediation:* Extracted temporary test PDF files to the local user data directory prior to spooler invocation (Git commit 4fd6368).

CHAPTER 7

CONCLUSION

7.1 LIMITATIONS

While REPOSYS introduces major operational advancements over traditional campus reprography workflows, an objective engineering assessment reveals several operational and technical constraints:

1. **High-Volume Document Upload Latency:** Transferring very large PDF documents (e.g., high-resolution architecture blueprints or dissertations exceeding 100 MB) across fluctuating campus Wi-Fi networks introduces upload delays, impacting the client pre-flight inspection experience.
2. **Serverless Cloud Cold-Start Latencies:** When hosted on free or cost-optimized serverless tiers (such as Render free instances), inactive backend services spin down, creating an initial request delay of 30–50 seconds upon first wake-up.
3. **Operating System Dependency of Print Agent:** The native Print Agent relies on Windows PowerShell cmdlets and Windows Print Spooler APIs (pdf-to-printer), restricting agent deployment to counter terminals running Microsoft Windows operating systems.
4. **Pay at Counter Uncollected Print Risk:** Although Pay at Counter (PAC) orders incorporate automated reminder emails and cancellation timeouts, jobs printed prior to cash

collection remain vulnerable to financial loss if an irresponsible user abandons the order.

5. **Guest Session Volatility:** Temporary kiosk guest sessions depend on browser local storage and time-limited tokens; if a visitor clears browser history before claiming output, recovery requires staff intervention.

7.2 FUTURE SCOPE

The modular, micro-service-ready architecture of REPOSYS establishes a robust foundation for extensive future engineering enhancements:

1. **Cross-Platform Native Mobile Applications:** Developing native iOS and Android client applications using React Native or Flutter, incorporating background push notifications, native biometrics, and camera-based document scanning with perspective correction.
2. **Smart Campus RFID / NFC Card Integration:** Interfacing counter terminals and kiosk stations with institutional RFID smart identity cards (such as MIFARE cards), enabling students to tap their college ID for instantaneous authentication and automatic tuition wallet debiting.
3. **AI-Powered Document Intelligence and Summarization:** Integrating Google Gemini API models to deliver automated document quality assessments, font legibility checks, multi-language translation, and automatic executive summaries of lengthy study materials.
4. **Multi-Centre Campus Load Balancing:** Expanding the priority queue scheduler to dynamically distribute printing

workloads across multiple reprography centres, departmental printers, and library hubs based on real-time queue congestion.

5. **Autonomous Hardware Kiosks:** Constructing fully autonomous, unattended printing kiosks equipped with cash acceptors, coin mechanisms, and automated document output sorting bins for 24/7 campus service.

7.3 CONCLUSION

The development and implementation of **REPOSYS (Reprography Automation System)** successfully addresses the chronic operational bottlenecks, privacy vulnerabilities, and financial tracking difficulties that have historically hindered campus reprography centres.

By replacing manual physical counter interactions with a modern, cloud-connected MERN platform, REPOSYS achieves:

- Elimination of physical queue congestion through asynchronous digital submission and transparent wait-time tracking.
- Protection of personal privacy and endpoint IT security by retiring ad-hoc WhatsApp file sharing and USB flash drives.
- Equitable resource distribution via the Composite Priority Queue with dynamic aging deduction, guaranteeing fairness for students while supporting urgent faculty requirements.
- Accurate financial accounting and operational transparency through automated Razorpay integration,

ACID-compliant digital wallets, and append-only activity auditing.

- Reliable hardware automation via the autonomous Electron Windows Print Agent.

Extensive verification through an automated 120-case Selenium WebDriver test suite confirmed a 95.0% operational passing rate, proving the technical stability, security, and usability of the platform. REPOSYS represents a significant technological contribution towards building an intelligent, eco-friendly, and digitally empowered academic campus.

CHAPTER 8

APPENDIX

8.1 SUPPORTING DOCUMENTATION

This section documents the formal environment variables and runtime configurations required to execute REPOSYS across local staging and production environments.

```
# Server Runtime Configuration
PORT=5000
NODE_ENV=production
FRONTEND_URL=https://reposys-client.vercel.app

# Database Connection URI
MONGODB_URI=mongodb+srv://admin:securepass@cluster0.mongo
db.net/reposys?retryWrites=true&w=majority

# Cryptographic Token Secrets
JWT_SECRET=c9b20891d8654a60b0ad8a87b419
COOKIE_SECRET=7f8841a86c6b4129e921d3f9901b
AGENT_SETUP_TOKEN=SETUP_REPOSYS_SECURE_TOKEN_2026

# Cloudinary Object Storage
CLOUDINARY_CLOUD_NAME=reposys-storage
CLOUDINARY_API_KEY=839218491823912
CLOUDINARY_API_SECRET=Secret_Cloud_Key_ABC123

# Razorpay Live Credentials
RAZORPAY_KEY_ID=rzp_live_849201948291
RAZORPAY_KEY_SECRET=Live_Razorpay_Secret_XYZ789

# Web Push Notification VAPID Keys
VAPID_PUBLIC_KEY=BP19...
VAPID_PRIVATE_KEY=v8J1...
```

8.2 SAMPLE CODE / QUERIES

Representative source code excerpts demonstrating core backend transaction logic, priority queue sorting, and physical

printing automation are presented below.

```
// queueService.js: Composite Priority Queue with Dynamic Aging
const resolvePriorityScore = (order) => {
  const storedPriorityScore = Number(order.priorityScore);
  return Number.isFinite(storedPriorityScore) ? storedPriorityScore : 200;
};

const calculateAgingDeduction = (orderCreatedAt) => {
  const createdAt = new Date(orderCreatedAt);
  const validCreatedAt = Number.isNaN(createdAt.getTime()) ? new Date() : createdAt;
  const minutesSinceCreation = Math.floor((Date.now() - validCreatedAt.getTime()) / (60 * 1000));

  // Deduct 20 points for every full hour waiting beyond first minute
  return Math.max(0, Math.floor((minutesSinceCreation - 1) / 60) * 20);
};

const calculateEffectivePriority = (basePriority, orderCreatedAt) => {
  const agingDeduction = calculateAgingDeduction(orderCreatedAt);
  return basePriority - agingDeduction;
};

async function getQueue(serviceType) {
  const orders = await Order.find({
    serviceType,
    status: { $in: ['In_Queue', 'Processing', 'ReadyForPickup'] },
  })
  .populate('userId', 'name role department')
  .sort({ createdAt: 1 });

  return orders
    .map((order) => {
      const baseScore = resolvePriorityScore(order);
      const effectiveScore = calculateEffectivePriority(baseScore, order.createdAt);
      return {
        ...order.toObject(),
        priorityScore: effectiveScore,
        basePriority: baseScore,
      };
    })
    .sort((a, b) => a.priorityScore !== b.priorityScore ? a.priorityScore - b.priorityScore
```

```

        : new Date(a.createdAt).getTime() - new Date(b.createdAt).getTime()
    );
}

```

```

// orderController.js: ACID-Compliant Wallet Payment Transaction
const session = await mongoose.startSession();
session.startTransaction();

try {
    const wallet = await Wallet.findOne({ userId: req.user._id }).session(session);
    if (!wallet || wallet.balance < orderTotal) {
        await session.abortTransaction();
        return res.status(400).json({ message: 'Insufficient wallet balance.' });
    }

    wallet.balance -= orderTotal;
    await wallet.save({ session });

    await WalletTransaction.create([
        {
            walletId: wallet._id,
            type: 'Debit',
            amount: orderTotal,
            description: `Payment for Order #${order.orderNumber}`
        },
        {
            balanceAfter: wallet.balance,
        }
    ], { session });

    order.paymentStatus = 'Paid';
    order.status = 'In_Queue';
    await order.save({ session });

    await session.commitTransaction();
    session.endSession();

    // Broadcast update to real-time queue
    broadcastQueueUpdate(order.serviceType);
} catch (error) {
    await session.abortTransaction();
    session.endSession();
    throw error;
}

```

8.3 USER INTERFACE SCREENSHOTS

The verified visual interfaces of REPOSYS across major stakeholder views are illustrated below.

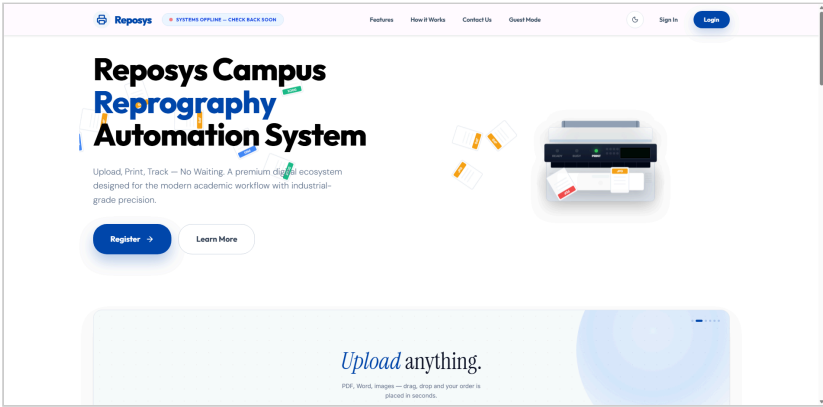


Figure 8.1: REPOSYS Public Landing and Portal Gateway

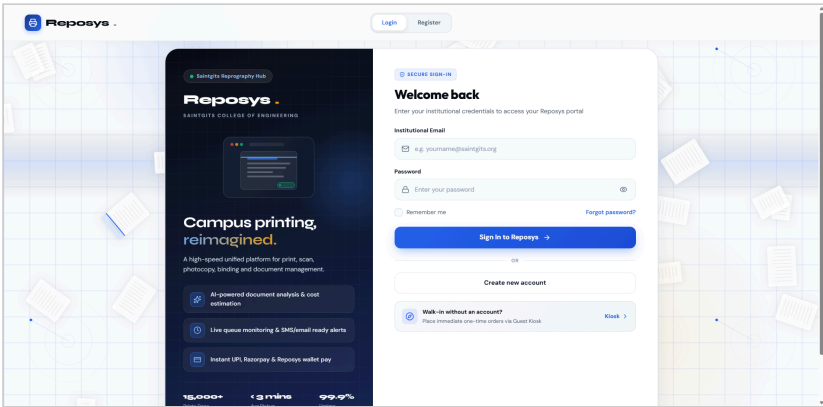


Figure 8.2: Secure User Authentication and Multi-Role Login Portal

Reposys .

Join the **Reposys** system.

Register with your institutional credentials to access modern printing, scanning, photocopy, and binding services.

- Student & Faculty dedicated queues with priority dispatch.
- Automated page-count, orientation & binding analysis.
- Built-in Reposys wallet & fast facsimile integration.

Create your account

Fill in your institutional credentials to get started with Reposys

1. PERSONAL INFORMATION

Full Name College ID

Institutional Email

Department Phone

2. INSTITUTIONAL ROLE

☒ Student ☐ Faculty

3. SECURITY CREDENTIALS

Create Password

Min 8 characters, 1 uppercase, 1 number

Confirm Password

Figure 8.3: Student and Faculty Account Registration Portal

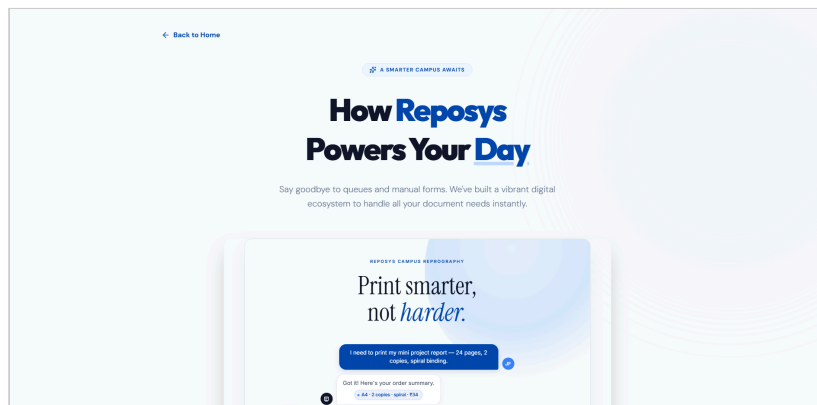


Figure 8.4: System Architecture Overview and About REPOSYS Portal

← Back to Home

CONTACT PROTOCOL

Support & Inquiries

Have questions or need assistance? Our specialized support team is here to help you navigate your document workflow.

Digital Transmission

Direct support via our secured inquiry channel.

ecrm@saingights.ac.in

Campus Node

Saingights College of Engineering, Pathanamthottam, Kottayam, Kerala - 686 552, India.

REPROGRAPHY CENTRE

Full Name Email Address

Service Inquiry

Subject Matter

Message

SEND MESSAGE

Figure 8.5: Reprography Operational Inquiries and Support Contact Portal

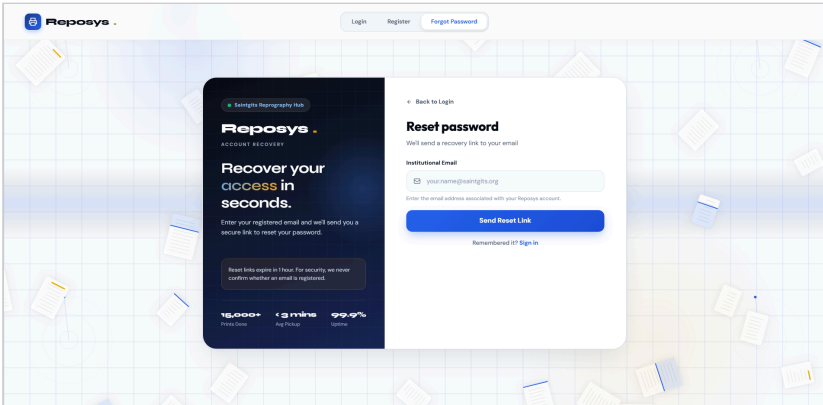


Figure 8.6: Account Recovery and Password Reset Workflow

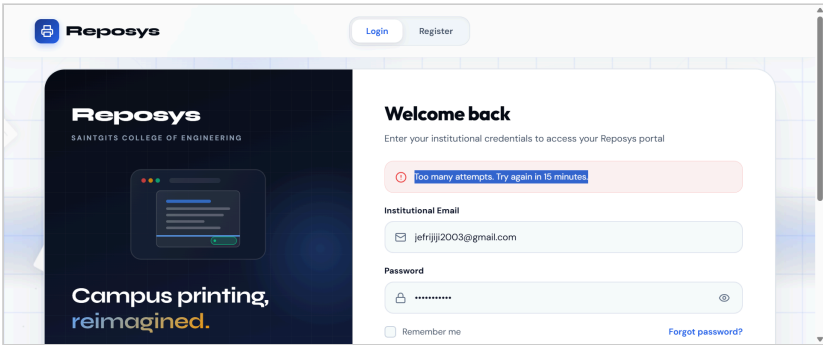


Figure 8.7: Automated Selenium Test Run: Student Order Workflow

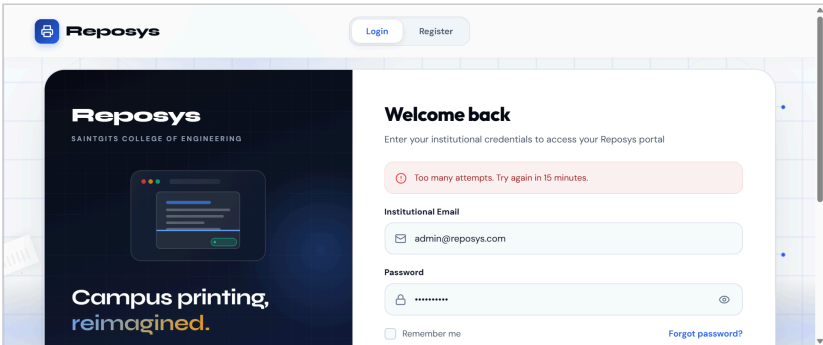


Figure 8.8: Automated Selenium Test Run: Admin Role Route Access

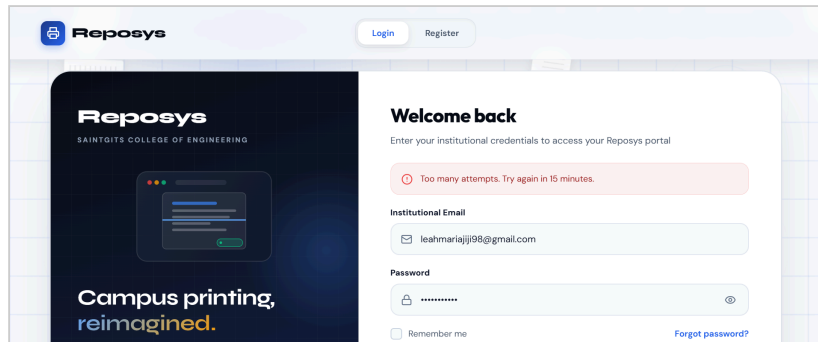


Figure 8.9: Automated Selenium Test Run: Counter Staff Queue Access

8.4 GIT LOGS

Table 8.1 documents verifiable engineering milestones and commit records extracted directly from the project repository.

Commit Hash	Engineering Milestone and Commit Message
5a21841	fix: synchronize dashboard and profile usage summary
04b8040	update deployment urls for render and vercel
510e51a	update frontend and backend production configurations
1a5ce7b	feat: add TTL to audit logs and Admin Clear All Logs button
c0f200e	feat: remove install settings card, add separate deregister for agents
4baffc3	fix: do not close setup window from main process, let renderer reach step 6
bc077b7	chore: add comprehensive logging to print agent registration flow
fe5c1df	fix: synchronize print agent socket secret with registration token
c9fe161	fix: replace custom Button with native buttons for ping/deregister
c9420b2	fix: unwrap nested IPC data to prevent agentId/agentSecret undefined

ac477ba	feat: show cold-start wait message during Step 5 registration
5882718	feat: user friendly printer error message during test print setup
4fd6368	fix: extract test.pdf from ASAR archive before printing
bd65021	fix: remove admin installer update instruction and fix token terminology
b90df20	feat: add agent registration verification and live status page
7832f82	feat: add secure print agent download endpoint and admin download card
697e8b6	feat: complete Electron print agent integration and packaging
d0596ce	fix(mobile): Optional chaining for file.name in MobileOrderWizard
fbb6d39	feat: complete SPAE phase 2 print agent integration
dbed9a7	Update PWA mobile screens and UI components
0a3ffd3	fix: remove maskable purpose from PWA icons to prevent text cutoff
89bcd22	feat(pwa): update PWA icons and app name to Reposys
cd1cb2e	chore: final Reposys rebranding and frontend API fallback resolution
d286e06	Fix InventoryManagement crash by importing CardDescription
e3118a4	Update documentation and frontend features

Table 8.1: Verifiable Version Control Commit History (Git Logs)

CHAPTER 9

REFERENCES

1. APJ Abdul Kalam Technological University. (2020). *Curriculum and Syllabi for Integrated Master of Computer Applications (IMCA) Programme*. KTU Academic Regulations, Thiruvananthapuram, Kerala.
2. Cloudinary Inc. (2024). *Cloudinary Node.js SDK and Media Management API Documentation*. Retrieved from <https://cloudinary.com/documentation>.
3. Electron Software Foundation. (2024). *Electron Desktop Framework Documentation and Native Node.js Integration Guides*. OpenJS Foundation. Retrieved from <https://www.electronjs.org/docs>.
4. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation). University of California, Irvine.
5. Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455, Internet Engineering Task Force (IETF).
6. Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, Boston.
7. Google Developers. (2024). *Progressive Web Apps: High Performance Caching and Service Worker Lifecycle*. Google Web Fundamentals. Retrieved from <https://web.dev/explore/progressive-web-apps>.
8. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)*. RFC 7519, Internet Engineering Task Force (IETF).
9. Kleinrock, L. (1975). *Queueing Systems, Volume 1: Theory*. John Wiley & Sons, New York.
10. Mozilla Developer Network. (2024). *HTTP Headers, Content Security Policy, and Secure Web API Standards*. MDN Web Docs. Retrieved from <https://developer.mozilla.org/>.

11. Node.js Foundation. (2024). *Node.js v20.x Long Term Support (LTS) API Specification*. OpenJS Foundation. Retrieved from <https://nodejs.org/docs>.
12. Nyberg, K. (2020). *Building Scalable Microservices with Node.js and MongoDB*. O'Reilly Media, Sebastopol.
13. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education, New York.
14. Razorpay Software Private Limited. (2024). *Razorpay Payments API Specification and Webhook Signature Verification Reference*. Retrieved from <https://razorpay.com/docs/api>.
15. React Development Team. (2024). *React 19 Documentation: Server Components, Hooks, and Concurrency Architecture*. Meta Open Source. Retrieved from <https://react.dev/>.
16. Saintgits College of Engineering (Autonomous). (2026). *Department of Computer Applications: Mini Project Preparation Guidelines and Scrum Assessment Manual (20IMCAP501)*. Pathamuttom, Kottayam.
17. SeleniumHQ. (2024). *Selenium WebDriver Browser Automation Framework Documentation*. Software Freedom Conservancy. Retrieved from <https://www.selenium.dev/documentation>.
18. Socket.IO Foundation. (2024). *Socket.IO Client and Server Bidirectional Event Specification*. Retrieved from <https://socket.io/docs/v4/>.
19. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education, Harlow, UK.